ПРАКТИЧЕСКАЯ РАБОТА № 2

Аудит готового клиент-серверного приложения

Выполнил: Макаров Семён

Группа: 23 ИС 41

Отчёт по проверке клиент-серверного приложения на Python

1. Ответы на вопросы по коду

Вопрос 1. Какой протокол используется в send_reliable?
Используется TCP. Это видно по созданию сокета с типом `socket.SOCK_STREAM` и вызову `connect()`, который устанавливает соединение с сервером. TCP обеспечивает надёжную и упорядоченную передачу данных.

Вопрос 2. Какой протокол используется в send_fast?
Используется UDP. В коде создаётся сокет типа `socket.SOCK_DGRAM`, а данные отправляются методом `sendto()` без предварительного установления соединения. UDP не гарантирует доставку пакетов, поэтому в клиенте предусмотрен тайм-аут.

Вопрос 3. В каком формате передаются данные?
Данные передаются в формате JSON, закодированном в UTF-8. Это подтверждает строка `json.dumps(message, ensure_ascii=False).encode("utf-8")` в файле `client.py`. Сервер декодирует байты и разбирает JSON с помощью `json.loads()`.

Вопрос 4. Что произойдёт, если отправить строку hello вместо JSON?
Сервер попробует разобрать полученные данные как JSON, но строка `hello` без кавычек не является корректным JSON. Возникнет `json.JSONDecodeError`; сервер выведет сообщение об ошибке и отправит ответ `{"status": "ERROR", "reason": "invalid JSON"}`.

2. Запуск клиента и сервера

При корректном запуске сервер должен вывести сообщения о запуске TCP на localhost:9000 и UDP на localhost:9001. После запуска клиента ожидаются ответы со статусом OK: для TCP — протокол TCP и формат JSON, для UDP — протокол UDP и формат JSON. В консоли сервера должны появиться полученные данные о погоде: город Москва, температура −5, влажность 80 и признак снегопада True.

Примечание. В рамках подготовки этого отчёта запуск программы и создание реальных скриншотов не выполнялись. Описание выше отражает ожидаемый результат, приведённый в задании; при сдаче следует добавить собственные скриншоты окон терминала после запуска.

3. Проверка граничных случаев

Случай 3.1. Сервер выключен, клиент TCP

При попытке выполнить `client.connect((HOST, PORT_R))` соединение не установится. Обычно возникает ошибка `ConnectionRefusedError`, поскольку на порту 9000 нет работающего сервера. В коде нет обработки этой ошибки, поэтому выполнение клиента прерывается, а UDP-часть не запускается.

Случай 3.2. Сервер выключен, клиент UDP

Если запустить только UDP-часть клиента, он отправит датаграмму, но ответа не получит. После тайм-аута в 2 секунды функция вернёт `{"status": "TIMEOUT"}` и измеренное время ожидания, близкое к двум секундам. UDP не устанавливает соединение и не сообщает отправителю, что сервер недоступен.

Случай 3.3. Клиент отправляет не JSON

Если вместо сериализованного объекта отправить байты `b"hello"`, сервер не сможет разобрать сообщение как JSON. В TCP-режиме он напечатает сообщение об ошибке и вернёт JSON-ответ со статусом ERROR и причиной invalid JSON; UDP-обработчик реализует аналогичную проверку. После эксперимента исходную строку отправки нужно восстановить.

Случай 3.4. Сравнение JSON и XML

JSON обычно компактнее в этом примере, потому что содержит пары «ключ — значение» без повторяющихся открывающих и закрывающих тегов. XML использует дополнительные теги для каждого поля и корневой элемент, поэтому строка XML получается длиннее. Кроме того, при восстановлении данных из XML значения в приведённом коде становятся строками: например, температура будет строкой "-5", а логическое значение — "True". Поэтому восстановленный словарь XML не полностью эквивалентен исходному объекту по типам данных.

4. Итоговый вывод

Для передачи погодных данных в этом приложении я бы выбрал TCP и JSON. TCP подходит, когда важна надёжная доставка запроса и ответа, а JSON удобен для обмена структурированными данными и обычно имеет небольшой размер. UDP может быть полезен, если важнее скорость и допустима потеря отдельных сообщений, однако в текущем примере TCP проще использовать для гарантированного обмена. XML тоже подходит для структурированных данных, но в данном примере он более многословен.

В ходе аудита также выявлено ограничение: TCP-клиент не обрабатывает ошибки подключения, поэтому при выключенном сервере программа завершается с исключением. Для более устойчивой работы стоит добавить обработку сетевых исключений и отдельно проверять преобразование типов при восстановлении данных из XML.